

Programming Languages

10. Data Types
(Chapter 8)

Ji Eun Kim

Fall 2026

Topics

- Data types and type information
- Type system
- Simple types
- Type constructors
- Type equivalence
- Type checking
- Type conversion
- Polymorphic type checking

Data Types and Type Information

Data Types and Type Information

- **Type name** represents the **possible values** that a variable of that type can hold and **the way those values are represented internally**
- **Data type** (definition 1): **a set of values**
  - e.g., `int` in Java:

    ```
    int x;
    x ∈ Integers= [-2147483648, 2147483647]
    ```
- **Data type** (definition 2): **a set of values**, together with a **set** of **operations** on that values having certain **properties**, e.g.,
  - arithmetic operations on integers or reals
  - subscript operation [ ] on arrays
  - member selection operation on structure or record types

Why are data types important?

- **Example: z = x / y; (Java)**

```
int x, y;
x = 5;
y = 2;
z = x / y;
```

- **integer division**, x/y results in 2.
- int z: z = 2;
- double z: z = 2.0;

```
double x, y;
x = 5;
y = 2;
z = x / y;
```

- **floating-point division**, x/y results in 2.5
- int z: wrong!
- double z: z = 2.5;

Strongly-Typed Languages

Strongly-typed: (Ada, ML, Haskell, Java, Pascal)

- Most data type errors detected **at translation time**
- A few checked during execution and runtime error reported (e.g., subscript out of array bounds).

- Pros:
  - No data-corrupting errors can occur during execution. (i.e., no unsafe program can cause data errors.)
  - Efficiency (in translation and execution.)
  - Security/reliability
- Cons:
  - May reject safe programs (i.e., legal programs is a subset of safe programs.)
  - Burden on programmers, may often need to provide explicit type information.

Legal programs ⊂ Safe programs: Java examples

실제 type 은 맞지만 선언이 다른 경우

```
Object o = "hello";
String s = o;        // Compilation error: Object를 String에 대입할 수 없음
```

절대 실행 되지 않는 코드

```
if (false) {
    int n = "abc";   // Compilation error
}
```

모든 경우에 return하지만 compile는 모름

```
int sign(int n) {
    if (n >= 0) return 1;
    if (n < 0)  return -1;
}  // Compilation error : missing return statement
// double n=NaN 일 경우 unsafe!
```

=>

```
//legal
int sign(int n) {
    if (n >= 0) return 1;
     else return -1;
    // 구조만 봐도 모든 경로가 return
}
```

Weakly-typed and untyped languages

- **Weakly-typed language**: one that has **loopholes** that may allow unsafe programs
  - Example: C, C++
    - interoperability of integers, pointers, arrays.

      e.g., int *p = (int *)0x1000; // 컴파일러가 해당 주소에 실제로 유효한 정수 데이터가 있는지 검사할 수 없음

- **Untyped** (or **dynamically typed**) languages: languages without static type systems
  - All safety checking is performed **at execution time**
  - Code with type errors: slower execution time
  - Example: Scheme, smalltalk, script languages (Python, Ruby)
    - Python: ”1” + 1 #TypeError!

Design Trade-off: Security vs. flexibility

- A type system tries to maximize both flexibility *and* security
  - flexibility:
    - reduce the number of safe illegal programs &
    - reduce the amount of type information the programmer must supply.

- Flexibility, no explicit typing or static type checking
  vs.
- Maximum restrictiveness, static type checking

Simple Types

Simple Data Types

- Have no other structure than their inherent arithmetic or sequential structure (내부적으로 더 이상 분해할 수 없는 최소 단위의 데이터 형태)
  - e.g., `integer`, `double`, `character`, and `boolean`.
- Often directly supported in hardware.
  - machine dependency (e.g., ALU, FPU)
- Most predefined types (e.g., `int`, `double`, `short`, `long`, `unsigned int`) are simple types.
  - exceptions: e.g., *String* in Java (not simple type).
- Some simple types are not predefined
  - enumerated types
  - subrange types

Enumerated Types

Ordered* set**, whose elements are ***named and ***listed*** explicitly.

- Examples:

```
enum Color_Type {Red, Green, Blue};          ( C )
type Color_Type is (Red, Green, Blue);       ( Ada )
datatype Color_Type = Red | Green | Blue;    ( ML )
```

- Operations?
  - Successor and predecessor
- Internal representation is varied.
  - Ada: actual names
  - C: integer (0, 1, .. or defined by the programmer)

Ada Example

```
type Color_Type is (Red, Green, Blue);

x : Color_Type := Green;

x := Color_Type'Succ(x); -- Blue
x := Color_Type'Pred(x); -- Green
put(x);          -- prints GREEN
```

```
x := Color_Type'Succ(x);
x := Color_Type'Succ(x);
-- runtime error
```

- No assumptions about the internal representation of values
- Print the value name itself

C Example

```
#include <stdio.h>
enum Color {Red, Green, Blue}; // 0, 1, 2
enum NewColor {NewRed=3, NewGreen=2, NewBlue=2};
int main() {
    enum Color x = Green;        // x is 1
    enum NewColor y = NewBlue; // y is 2
    x++;     // 2 -- Blue
    y--;     // 1 -- not even in the enum
    printf("%d\n",x);  //2
    printf("%d\n",y);  //1
    return 0;
}
```

- **`enum`** in C is simply **`int`**
- Can **customize** the values

Java *enum* Example – not simple type, but class

```
public enum Planet { // static and final object
    MERCURY (2.4397e6), EARTH (6.37814e6);
    private final double radius; //
    Planet(double radius) { this.radius = radius; }
    private double radius() { return radius; }

    public static void main(String[] args) {
        for (Planet p : Planet.values())
            System.out.printf("The radius of  %s is
  %f\n", p, p.radius());
    }
}
```

```
The radius of MERCURY is 2439700.000000
The radius of EARTH is 6378140.000000
```

Evaluation of Enumeration Types

- **Efficiency** – e.g., compiler can select and use a compact efficient representation (e.g., small integers)
- **Readability** -- e.g. no need to code a color as a number
- **Maintainability** – e.g., adding a new color doesn't require updating hard-coded constants.
- **Reliability** -- e.g. compiler can check operations and ranges of value.

C Example for Maintainability

```
enum Color {White, Green, Blue, Black};
enum Color {White, Yellow, Green, Blue, Black};
int main(){
    enum Color x = Black;
    int i = x;
    while (i >= White){
        if (i < Green)
            printf("this is a light color!\n");
        i--;
    }
..
}
```

//initial, Green => 1
// added Yellow later, Green=>2

//Intention: green 보다 작으면 밝은 색

What if no enumeration?

```
   if (i < 1) printf("this is a light color!\n");
```

=> Has to be changed to:

```
   if (i < 2) printf("this is a light color!\n");
```

Ada Example for Reliability

```
type DAY is (MON, TUE, WED, THU, FRI, SAT, SUN);
type DIRECTION is (NORTH, EAST, SOUTH, WEST);

GOAL : DIRECTION;
TODAY : DAY;
START : DAY;

TODAY := MON;
GOAL := WEST;
START := TODAY;

TODAY := WEST; -- Illegal: WEST is not a DAY value at compile time
TODAY := 5; -- Illegal: 5 is not a DAY value
TODAY := TODAY + START; -- Illegal: "+" is not defined for DAYS
```

Subrange Types and Ordinal Types

- **Subrange types**: **contiguous subsets** of simple types specified by giving least and greatest elements
- Example in Ada:

```
type Digit_Type is range 0..9; //0, 1, 2, 3,…, 8, 9
```

- **Ordinal (서수) types**: types that exhibit a **discrete order** **on the set of values**
  - All numeric integer types are ordinal types
  - Always have comparison operators
  - Often have **successor** and **predecessor operations**
- **Real numbers are not ordinal**; they have no successor and predecessor operations

Type Constructors

Type Constructors: Defining New Types

- Type constructors as set operations:
  - Cartesian product (e.g., record, struct, tuple)
  - Union (e.g., (un)tagged union)
  - Subset (e.g., subrange)
  - Functions (Arrays)
- Some type constructors do not correspond to set operations (e.g., pointers, closer to memory structure)
- Some set operators don’t have corresponding type constructors (e.g., intersection)

Cartesian Product (데카르트 곱)

- **Ordered pairs of elements from U and V**

  $U \times V = \{(u, v) \mid u \in U \text{ and } v \in V\}$

- **Operations: projection**

  $p_1: U \times V \rightarrow U; \quad p_2 : U \times V \rightarrow V$

  $p_1((u,v))=u; \qquad p_2((u,v))=v$

- Cartesian Product is available as the **record** or **structure** in many programming languages.

Example: Cartesian Product of Type

- `struct` in C :

```
struct IntCharReal
 {     int i;
       char c;
       double r;
 };
```

int × char × double

- `record` in Ada

```
type IntCharReal is record
       i: integer;
       c: character;
       r: float;
 end record;
```

int × character × float

C Example: the same type?

```
struct IntCharReal
{     int i;
      char c;
      double r;
};
```

```
struct IntCharReal
{    char c;
      int i;
      double r;
};
```

C Example: the same type?

```
struct IntCharReal
{     int i;
      char c;
      double r;
}
```

```
struct IntCharReal
{    int j;
      char ch;
      double d;
}
```

Record/structure are not exactly Cartesian products

- Cartesian product vs. a record structure:
  - **Cartesian product:** they are referred to **by position**
  - **Record/struct structure**: components have **names**
- **Most languages** consider component **names to be part of the type** defined by a record structure
- Component selector: projection by **component names**

```
struct IntCharReal x;    x.i;
```

- Thus the previous two types can be considered **different**, even though they represent the same Cartesian product.

ML: Pure Cartesian Product

Tuple: a purer form of record structure in ML that is essentially identical to the Cartesian product

```
                              #1      #2      #3
type IntCharReal = int * char * real;//No name for type
```

- `(2, #"a", 3.14)`
- `#3(2, #"a", 3.14) => 3.14`
  - get the value of #3 position

Union

- $U \cup V = \{ x \mid x \in U \text{ or } x \in V \}$
  - Data items with **different types** are stored in overlapping region, reduce memory allocation.
  - Only one type of value is valid at one time.
- Unions are useful in reducing memory allocation requirements for structures when different data items are not needed simultaneously
- Two varieties:
  - **Discriminated unions***:* a **tag** or **discriminator** is added to the union to distinguish the type of its elements
  - **Undiscriminated unions***:* lack the tags
- A language with **undiscriminated** unions has an **unsafe** type system

Undiscriminated Union in C

- In C and C++, the union type constructor creates undiscriminated unions

```
union IntOrReal {
   int i;
   double r;
};

union IntOrReal x;

x.i = 1;

printf("%f\n", x.r); // print an unexpected value
```

- No tag/discriminator to distinguish which type the element is
- Can be **unsafe**

Create Discriminated Union in C++

```
struct IntOrReal {
   bool isInt;
   union {
       int i;
       double r;
   };
};

IntOrReal x;
x.isInt = true;
x.i = 1;
...
if (x.isInt) printf("%d\n", x.i);
else printf("%f\n", x.r);
```

- Safe now or not?

Discriminated Union in Ada

- Variant record (with tag or discriminator)

```
type Disc is (IsInt, IsReal);
type IntOrReal (which: Disc) is
record
   case which is
          when IsInt =>  i: integer;
          when IsReal =>  r: float;
   end case;
end record;
...
x: IntOrReal := (IsReal, 2.3);
put (x.i);  -- generates ERROR at execution time,
            -- x.which = IsReal at this point
```

Safe!: programmers won't be able to create inconsistent data

Subset

- U = { v | v satisfies certain conditions and v ∈ V}
- Ada subtype mechanism: by specifying subrange (lower and upper bound)
- Example 1
  - `type Digit_Type is range 0..9;`

completely new type
Integer와 연산 불가

vs.

  - `subtype IntDigit_Type is integer range 0..9;`

subtype
Integer와 연산 가능

subtype in Ada

- Example 2

```
type Disc is (IsInt, IsReal);
type IntOrReal (which: Disc) is record
   case which is
           when IsInt =>  i: integer;
           when IsReal =>  r: float;
   end case;
end record;

subtype IRInt is IntOrReal(IsInt);
subtype IRReal is IntOrReal(IsReal);

x: IRReal :=  2.3; -- compile error
x: IRReal := (IsReal, 2.3); -- OK
```

subset types inherit operations from their parent types.

Subset

- Subset types inherit operations from their parent types
  - Most languages do not allow the programmer to specify which operations are inherited and which are not
- **Inheritance** in object-oriented languages can also be viewed as a subtype mechanism
  - With a great deal more control over which operations are inherited

Arrays and Functions

- The set of all functions `f : U→V`
  - **Array type** or
  - **Function type**

개념	수학적 정의	프로그래밍
타입 f:U→V	집합 U에서 집합 V로의 모든 함수들의 집합	배열 타입 또는 함수 타입
Array	U가 서수 타입(ordinal type)일 때의 함수	인덱스(U)에 의해 컴포넌트/요소(V)를 찾는 매핑
Function	U를 입력으로, V를 출력으로 하는 매핑	매개변수(U)를 받아 반환 값(V)을 계산하는 로직

Array in C/C++

- Allocated on **stack** (for local array), size **statically** specified (at compile time).

```
typedef int TenIntArray [10];
typedef int IntArray []; // array type w/o size: OK
                         //(incomplete type, no memory allocation)

TenIntArray x;
int y[5];
int z[]={1,2,3,4}; // OK. size inferred from
                   // initializer:4
IntArray w = {1,2}; // OK. size become 2
IntArray v;   //illegal(block scope):size unknown
```

Array in C/C++

```
int n = ... //from user input
int a[n]; // VLA
```

- VLA (Variable Length Array) : 배열 크기를 실행 중에 정해서 지역 배열을 선언할 수 있게 컴파일러가 처리
  - C89/C++ : illegal (1989년), array size must be a constant
  - C99: legal (VLA, local arrays only)
  - C11(2011년) and later: optional feature

C version: C89 → C99 → C11 → C17 → C23

Array in C/C++

- dynamically defining an array size in C++:

```
int f(int size) {
   int a[size]; // C++: illegal / C99: legal / C11+: optional
}
```

Array in C/C++

- An array parameter can omit its size int array_max(int a[], int size)
  - **a** is actually a pointer `(int *a)`; only the address is passed
  - So the size must be passed separately
  - **`sizeof(a)`** gives the pointer size, not the array size

```
int array_max (int a[], int size){
    int temp;
     ...
 return temp;
}
```

Array in Java

- Allocated on **heap**, size **dynamically** specified;
- Size can be obtained by `.length` (a part of array information)

```
int n = ...   //from user input
int [] x = new int [n]; // not illegal
System.out.println(x.length);
```

Array in Ada

- size **dynamically** specified; allocation on **stack**
- Set of subscripts

```
type IntToInt is array(integer range <>) of integer;
n : Integer;

get(n);    -- from user input
declare
   x: IntToInt(1..n) :=(others =>0);
begin
   for i in x'range loop
      put(x(i));
   end loop;
end;
```

Ada vs. C

```ada
-- Ada
get(n);    -- from user input
declare
  x: IntToInt(1..n) :=(others =>0);
begin
   for i in x'range loop
      put(x(i));
   end loop;
end;
```

```c
/* C99 */
scanf("%d", &n);
{
    int x[n];
    for (int i = 0; i < n; i++)
        printf("%d", x[i]);
}
```

x[n] ?

Multi-dimensional arrays

- C/C++: `int x[10][20];`
- Java: `int [][] x = new int [10][20];`
- Ada: These two are different

```
type Matrix_Type is array(1..10, -10..10) of
  integer;
x(i,j); -- 2차원 배열

type Row is array(-10..10) of Integer
type Matrix_Type is array(1..10) of Row;
x(i)(j); -- 배열의 배열
```

Storage for Array (1/2)

- row-major form

  `x[1,-10],x[1,-9],…,x[1,10],x[2,-10],…,x[2,10],x[3,-10],…`

- column-major form

  `x[1,-10],x[2,-10],…,x[10,-10],x[1,-9],…,x[10,-9],x[1,-8],…`

- C/C++: must be **row-major** form

  ```
  int array_max(int a[][20], int size)
  ```

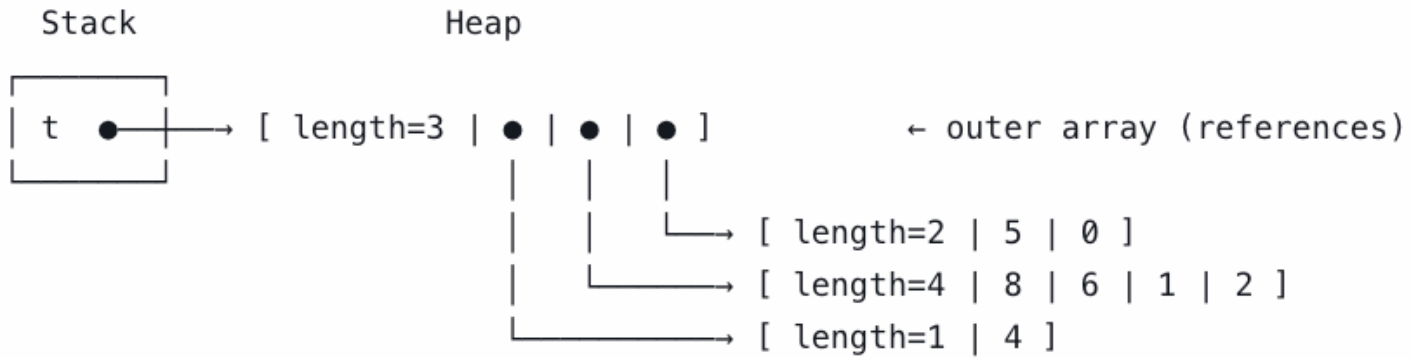
  - row-major form: The size of the 2nd dimension is required for compiler to compute the distance in memory between a[i] and a[i+1]

Storage for Array (2/2)

- Java

  `int array_max(int [][]a)`
  - 호출 시 배열을 넘기면 크기는 `.length`로 알 수 있음

```
int[][] t = new int[][] { {4}, {8, 6, 1, 2}, {5, 0} };
array_max(t);
```



Function Type in C

- General function and procedure types can be created in some languages.
- Example: in C, define a function type from integers to integers:
- define a function type with *int* parameter and *int* return

```
typedef int (*IntFunction)(int);
```

pointer to function

- Use this type for variables or parameters:

```
int square(int x) {return x*x;}
IntFunction f = square;
int evaluate(IntFunction g, int value){
    return g(value);
}
…
printf("%d\n",evaluate(f,3)); // square(3)
```

Function Type in ML

```
type IntFunction = int -> int;

fun square(x: int) = x * x;

val f = square;
fun evaluate(g:IntFunction, value:int) = g value;
…
evaluate(f,3); (*evaluates to 9 *)
```

함수 본문 시작

g를 value에 적용

Function: First class value

Vector, List

Functional languages: usually do not supply array

- Vectors: like arrays, more flexibility, especially dynamic resizability. (Scheme)
- Lists: like vectors, can only be accessed by counting down from the first element.

Pointer

- A *pointer type:* range of values consists of **memory addresses** and a special value, nil, or null
- Advantages:
  - Addressing **flexibility** (address arithmetic, explicit dereferencing and address-of operator, domain type not fixed (void *))
  - Dynamic storage management e.g.,
    - `malloc /new, free/delete`
  - Recursive data structures, e.g., linked list

ptr+1

*ptr

&var

```
struct CharListNode{
    char data;
   struct CharListNode* next;
};
typedef struct CharListNode* CharList;
```

Pointers and Recursive Types

- **Reference** or **pointer** constructor: constructs the set of all addresses that refer to a specified type
- Example in C: `typedef int* IntPtr;`
  - Constructs the type of all addresses where integers are stored
- Pointers are **implicit** in languages that perform automatic memory management
  - In Java, all objects are implicitly pointers that are allocated explicitly (using the `new` operator) but deallocated automatically by garbage collection

Pointers and Recursive Types (cont'd.)

- **Reference**(&): **address** of an object **under control of the system** that cannot be used as a value or operated on in any way (except copying)
  - Alias of an object
- **Pointer**: can be used as a value and **manipulated** by the programmer
- References in C++ are created by a **postfix & operator**

```
double r= 2.3;
double& s = r;  // s is a reference to r. C++ only
s +=1;
cout << r << endl; // prints 3.3
```

Pointer and Array

- Array are **implicitly constant pointers** to their first component.

```
int a[] = {1,2,3,4,5};  // a[0] = 1, etc.
int* p = a;             // p points to first component of a
printf("%d\n", *p);     // prints value of a[0], so 1
printf("%d\n", *(p+2)); // prints value of a[2]= 3
printf("%d\n", *(a+2)); // also prints 3
printf("%d\n", 2[a]);   // also prints 3!
```

a[2] : shortcut of *(a +2)
by the commutative addition *(a + 2)

Recursive Type

- **Recursive type**: a type that uses **itself in its declaration**

```
struct CharList{
    char data;
    struct CharList next; // not legal in C!
};
```

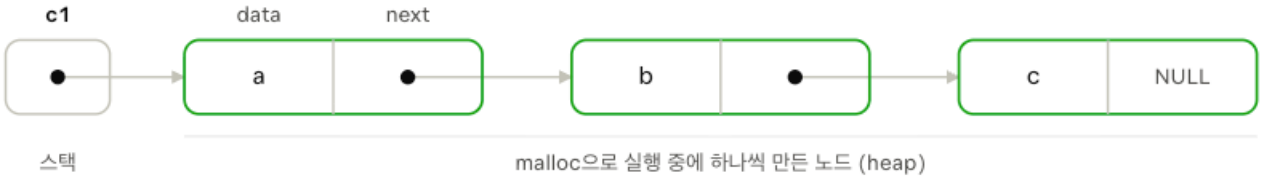


- C's data allocation strategy: each data type should have **a fixed maximum size** determined **at translation time**.
- Use **pointer** to allow *manual* dynamic allocation

Recursive Type (Cont'd)

```
struct CharListNode{
    char data;
    struct CharListNode* next; //now legal in C!
};
typedef struct CharListNode* Charlist;
Charlist c1 = (Charlist) malloc(sizeof(struct
CharListNode));
(*c1).data = 'a';
(*c1).next = 0;
```

합법: next를 포인터로 바꾸면 크기가 고정됨



Type Equivalence

Type Equivalence

- How to decide if two types are the same?

- **Structural equivalence**: two data types are the same if they have **the same structure**
  - Built in the same way **using the same type constructors** from the same simple types
- **Name equivalence**: two types are the same **only if** they have **the same name**
  - **Two variables** are type equivalent only if **their declarations use exactly the same type name**

Structural Equivalence

- Example:
  - `Rec1` and `Rec2` are structurally equivalent
    - `char` X `int` X `char[10]`
  - `Rec1` and `Rec3` are not structurally equivalent
    - `char` X `int` X `char[10]` vs.
    - `int` X `char` X `char[10]`

```
struct Rec1{
    char x;
    int y;
    char z[10];
};

struct Rec2{
    char x;
    int y;
    char z[10];
};

struct Rec3{
    int y;
    char x;
    char z[10];
};
```

Structural Equivalence

- **How much information is included in a type** under the application of a type constructor?
- Example: are `A1` and `A2` structurally equivalent in a language?
  - `typedef int A1[10]`
  - `typedef int A2[20]`

  – Yes, if **size of the index** set is **not part of an array type**
  – Otherwise, no

Type Equivalence

- Similar question arises regarding **member names** of structures
- Example: Are these two structures structurally equivalent?

```
struct RecA{
    char x;
    int y;
};
```

```
struct RecB{
    char a;
    int b;
};
```

  - If structures are considered to be **just Cartesian products**, then yes
  - They are ***typically*** **not** considered equivalent in most languages, because variables of different structures would have to use **different names to access member data** (RecA: .x, RecB:.a)

Type Equivalence

- **Type names** in declarations may or may not be given explicitly
  - In C, variable declarations can use **anonymous types**
  - Names can also be given right in `structs` and `unions`, or by using a `typedef`
- Structural equivalence when type names are present can be done by simply replacing each name by its associated type expression in its declaration (except for recursive types)

```
typedef struct { int a; } S;
S var1;  // replace S -> struct { int a; }
```

Name Equivalence

- **Name equivalence**: two types are the same **only if** they have **the same name**
  - Easier to implement than structural equivalence, as long as every type has an explicit name
  - **Two types** are equivalent only if they are **the same name**
    - assume the structure of those two types should be same.
  - **Two variables** are type equivalent only if **their declarations use exactly the same type name**

Type Equivalence (cont'd.)

- Example: in **C "style"** code
  - Variable `a` has two names: `struct RecA` and `RecA` (given by the `typedef`)
  - Variable `b` has only the name `RecB` (the `struct` name was left blank)
  - Variable `c` has no type name at all (only an internal name not usable by the programmer)

```
struct RecA{
    char x;
    int y;
} a;

typedef struct RecA RecA;

typedef struct{
    char x;
    int y;
} RecB;

RecB b;

struct{
    char x;
    int y;
} c;
```

Type Equivalence

- Example: in C “style” code:
  - `a`, `b`, `c`, and `d` are structurally equivalent?
  - `a` and `c` are (pure) name equivalent, and not name equivalent to `b` or `d`?
  - `b` and `d` are not (pure) name equivalent to any other variable?

- Note: in C

```
struct RecA{
    char x;
    int y;
};

typedef struct RecA RecA;

struct RecA a;
RecA b;
struct RecA c;
struct{
    char x;
    int y;
} d;
```

Type Equivalence Rules in Various Languages

- Ada: Pure form of **name equivalence**
  - Requires type names in variable and function declarations in virtually all cases
- C
  - **Name equivalence** for `struct` and `union`
  - **Structural equivalence** for everything else
  - typedef does not create a new data type (but just alias).
    - precisely, C uses **declaration equivalence** (two types are equivalent if they are derived from the same type name)
- Pascal
  - Similar to C, except that almost all type constructors lead to new, inequivalent types

Type Equivalence (cont'd.)

- Java's approach is simple:
  - It has no `typedefs`
  - **`class`** and **`interface`** declarations implicitly create new type names, and **name equivalence** is used for these types
  - **Arrays** use **structural equivalence**, with special rules for establishing base type (e.g., int, double etc) equivalence

```
int[] x = new int[3];
int[] y = new int[100];
x = y;          // OK: both int[]
```

```
int[]     p = {1, 2, 3};
Integer[] q = {1, 2, 3};
p = q;          // Compilation error
q = p;          // Compilation error
```

Type Checking

Type Checking

- Type Checking: Determine whether the program is correct in terms of data types.
  - Type Inference: Types of expressions
  - Type Equivalence: Are two types the same?
  - Type Compatibility: Relaxing exact type equivalence under certain circumstances
    - E.g., using int in double type variable

Example

```
long y;
float x;
double c;
x = y/2 + c;
```

- y is long, 2 is int, so promoted to long, y/2 long
- c is double, y/2 is long, so promoted to double, y/2 + c is double
- x is float, y/2 + c is double, what happens?
  - C? warning or OK
  - Java? compile error

Example: C

```
struct RecA {
   int i;
   double r;
};

int p(struct {int i; double r;} x)
{ ... }

int q(struct RecA x)
{ ... }
```

```
struct RecA a;
int b;

b = p(a); //type error!
b = q(a); //legal
```

structural equivalent but not name equivalent:

Implicit Types

- **Implicit types**: types that are **not explicitly given in a declaration**
  - The type must be **inferred by the translator**, either from **context information** or from standard **rules**
- In C:
  - Note: since C99, implicit int is removed; every declaration needs a type.
    - Older compilers only warn; GCC 14+ and Clang 16+ report an error.
    - unsigned x; long y; are still legal: int is just omitted.
- In Pascal named constants are implicitly typed by the literals they represent
- Literals are the major example of implicitly typed entities
  - e.g., 10 : int, 3.14: double, ‘A’: **int** (in C), char in C++ and Java

Implicit Types

- **Implicit types**: types that are **not explicitly given in a declaration**
  - The type must be **inferred by the translator**, either from **context information** or from standard **rules**
- In C:
  - Variables are implicitly integers if no type is given,
  - Functions implicitly return an integer value if no return type is given
  - Note: since C99, implicit integer (omitting the type in a declaration) is illegal. Literals and expressions are still implicitly typed.
- In Pascal named constants are implicitly typed by the literals they represent
- Literals are the major example of implicitly typed entities
  - e.g., 10 : int, 3.14: double, 'A': **int** (in C), char in C++ and Java

Overlapping Types and Multiply-Typed Values

- Two types may overlap, with values in common
- Although preferable for types to be disjoint, this would eliminate the ability to create subtypes through inheritance in object-oriented languages

- In C, types like `unsigned int` and `int` overlap
- In C, the literal `0` is a value for every integral type, a value of every pointer type, and represents the null pointer
- In Java, the literal value `null` is a value of every reference type

Shared Operations

- Each type is associated, usually implicitly, with a set of operations
- Operations: shared among several types or have the same name as other operations that may be different
- Example: + operator can be real addition, integer addition, or set union
  - Integer addition: ALU vs. real addition: FPU
- Overloaded operation: the same name is used for different operations
  - Translator must decide which operation is meant based on the types of the operands

Type Conversion

Type Conversion

- **Type conversion**: converting from one type to another
  - Can be built into the type system to happen automatically
- **Implicit conversion** (or **coercion**): inserted by the translator
- **Widening conversion**: target data type's range covers the source type
  - No overflow, but some may lose precision

```
//Java
int   i = 16777217;   // 2^24 + 1
float f = i;          // widening, compiles without cast
                      // f == 16777216.0 → precision lost
```

- **Narrowing conversion**: conversion may involve a loss of data

Type Conversion

- **Narrowing conversion**: the target type's range may not cover the source type
  - values can overflow or be cut off, as well as lose precision
    - (e.g., double → int, int → byte)

```
// C
int a = 3.7;                  / / 3 : fraction cut off (toward zero)
float f = 0.1;                // 0.100000001490116... : precision lost
int c = 1e10;                 // undefined behavior: out of int range

//Java: use explicit casting
int  a = (int) 3.7;           // 3  : fraction cut off , a =3.7 compilation error
float f = 0.1f;               // or f = (float) 0.1; f = 0.1 로 하면 compilation error
int  c = (int) 1e10;          // 2147483647 : out of range, clamped to max
```

Example

```
long y;
float x;
double c;
x = y/2 + c;
```

- `y` long, `2` is int, so promoted to long, `y/2` long.
- `c` is double, `y/2` is long, so promoted to double, `y/2 + c` is double.
- `x` Is float, `y/2+c` is double, what happens?
  - C?
  - Java?

Type Conversion

- **Implicit conversion**:
  - Can weaken type checking so that errors may not be caught
  - Can cause unexpected behavior if the conversion is done in a different way than the programmer expects

- **Explicit conversion** (or **cast**): conversion directives are written into the code
  - Conversions are documented in the code
  - Less likelihood of unexpected behavior
  - Makes it easier for the translator to resolve overloading

Type Conversion (cont'd.)

- Example In C++:

```
double max (int, double);
double max (double, int);
max (2, 3)
```

  - **Ambiguous**, because of the possible implicit conversions from `int` to `double` on either first or second parameter

- Java only permits widening implicit conversions for arithmetic types
- C++ emits warning messages for narrowing

Type Conversion

- **Explicit** casts need to be somewhat restricted
  - Often to **simple types**, or just **arithmetic types**
- If casts are permitted for structured types (e.g., struct, record), they must have identical sizes in memory
  - Allows translation to **reinterpret** the memory as a different type
  - causes memory corruption or runtime error
- Example: in C, `malloc` and `free` functions are declared using a **generic pointer** or **anonymous pointer type** `void*`
- Object-oriented languages allow conversions from subtypes to supertypes (upcasting) and back (downcasting) in some cases

Example: Java

- **Implicit** conversion:
  - Representation change
    - type promotion, e.g., `int` to `double`
  - No representation change (객체는 그대로, 참조 타입만 바뀜)
    - upcasting (subtype to super type)

```
SubA s = new SubA();
A a = s; // implicit upcasting
```

- **Explicit** conversion:
  - Representation change

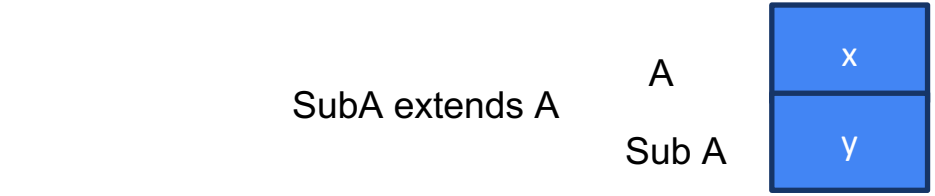
```
double x = 1.5; // x: 0x3FF8000000000000 (64bits)
int y = (int)x; // y: 0x00000001(32비트)
```

  - No representation change
    - downcasting (super type to subtype)

```
A a = new SubA();
SubA s = (SubA) a;
// 객체는 그대로, 참조의 타입만 바뀜
```

Java

SubA extends A




```
SubA s = new SubA();
A a = s;        // OK: implicit upcasting
```

```
A a = new SubA();

a.x;                    // OK: A에 x가 있음
a.y;                    // 컴파일 에러: 컴파일러는 a를 A로만 봄
a.getClass();           // class SubA: 실행 중 실제 객체 타입
((SubA) a).y;           // OK: 캐스트로 SubA로 보겠다고 알려 줌, 실행 중 검사 통과
```

```
A a = new A();
SubA s = (SubA) a;  //  explicit downcasting, 컴파일 OK,
                    //  runtime에  ClassCastException error
```

Type Conversion

- Alternative to casts is to use **predefined** or **library functions** to perform conversions (safe)
  - Ada uses **attribute functions** to allow conversions
    - Example: Integer 'Image(X)
      - Integer: type, Image: attribute, convert integer to string representation
  - Java contains functions e.g.
    - `Integer.toString` to convert from `int` to `String`
    - `String.parseInt` to convert from `String` to `int`
- Undiscriminated unions can hold values of different types
  - With no discriminant or tag, a translator cannot distinguish values of one type from another
  - No type safety

Polymorphic Type Checking

Polymorphic Type Checking

- Most **statically typed languages** required that **explicit type information** be given for all names in **declarations**
- It is possible to determine types of names without explicit declaration:
  - Can collect information on the uses of a name and infer the type from the set of all uses
  - Can declare a type error because some of the uses are incompatible with others
- This type inference and type checking is called **Hindley-Milner type checking**
  - Functional PL (ML, Haskell)

Conventional Type Checking

- Example in C code: `a[i] + i`
  - `a` must be declared as an array of integers, and `i` as an integer, giving an integer result
- Type checker starts out with this syntax tree:

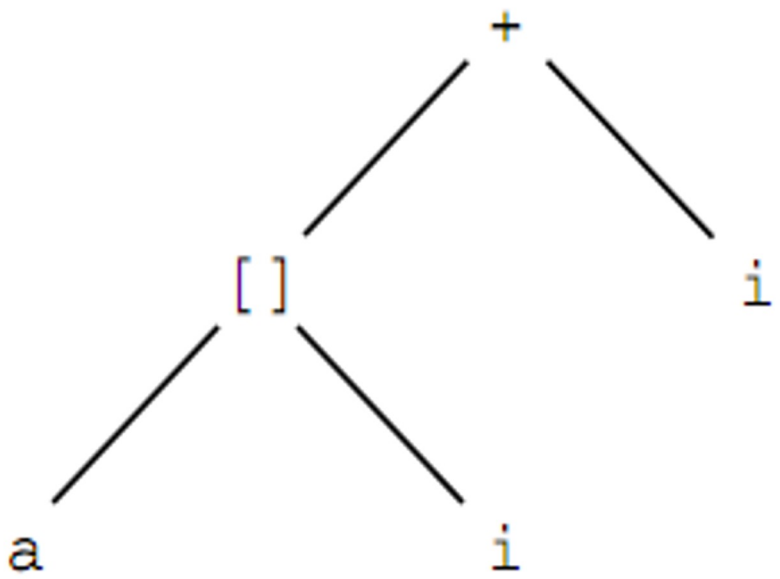


Figure 8.8 Syntax tree for a[i] + i

Conventional Type Checking (cont'd)

- Types of the names (leaf nodes) are filled in from **declarations**

```
int a[10];
int i;
...
a[i] + i
```

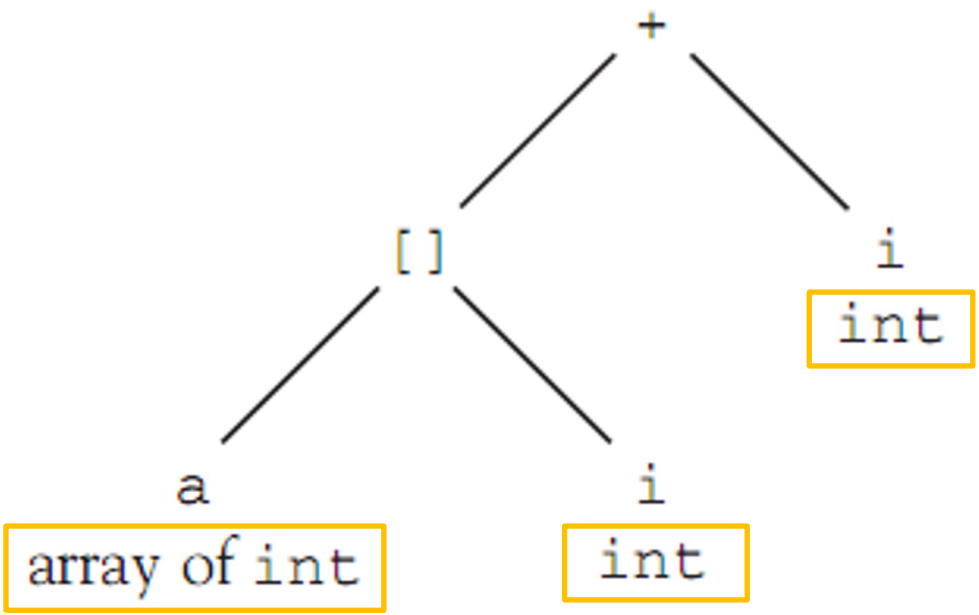


Figure 8.9 Types of the names filled in from the declarations

Conventional Type Checking (cont'd)

- Type checker now checks the subscript node (labeled `[]`)
  - Left operand must be an array
  - Right operand must be an `int`
  - Inferred type of the subscript node is the component type of the array - `int`

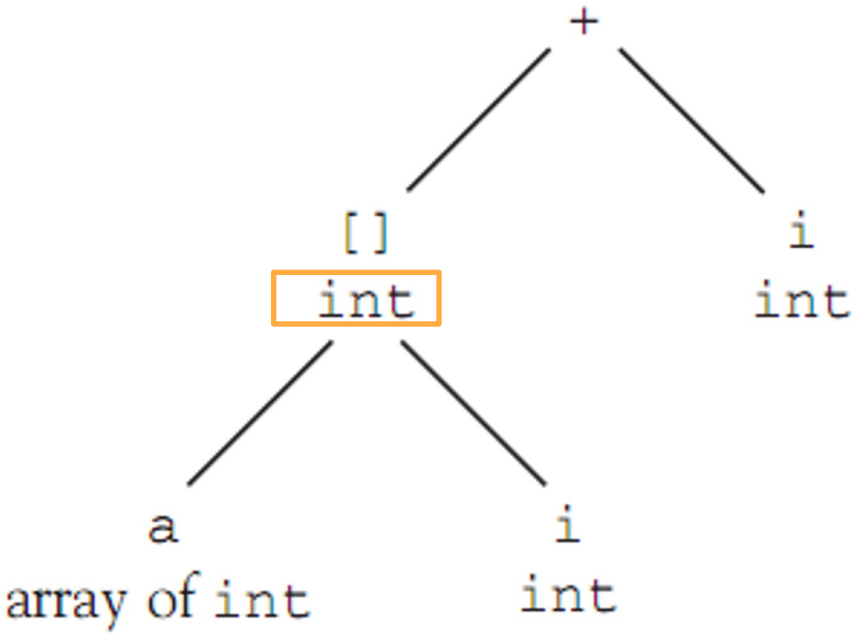


Figure 8.10 Inferring the type of the subscript node

Conventional Type Checking (cont'd)

- `+` node type is checked
  - Both operands must have the same type
  - This type must have a `+` operation
  - Result is the type of the operands - `int`

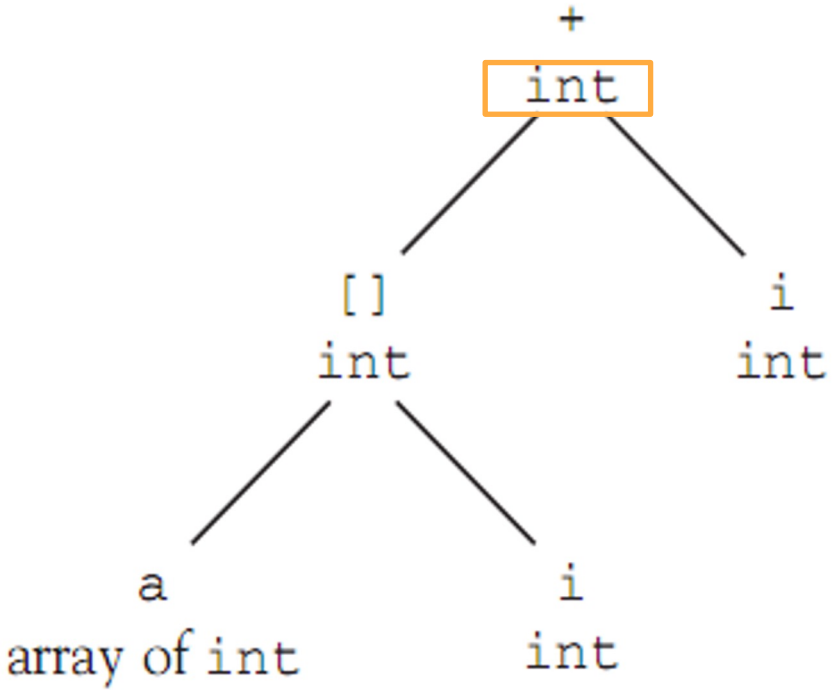


Figure 8.11 Inferring the type of the + node

Hindley-Milner Type Checking

- Example:

  `a[i] + i`

  – What if the declarations of `a` and `i` were missing?

- Type checker would first **assign type variables to all names** that do not yet have types

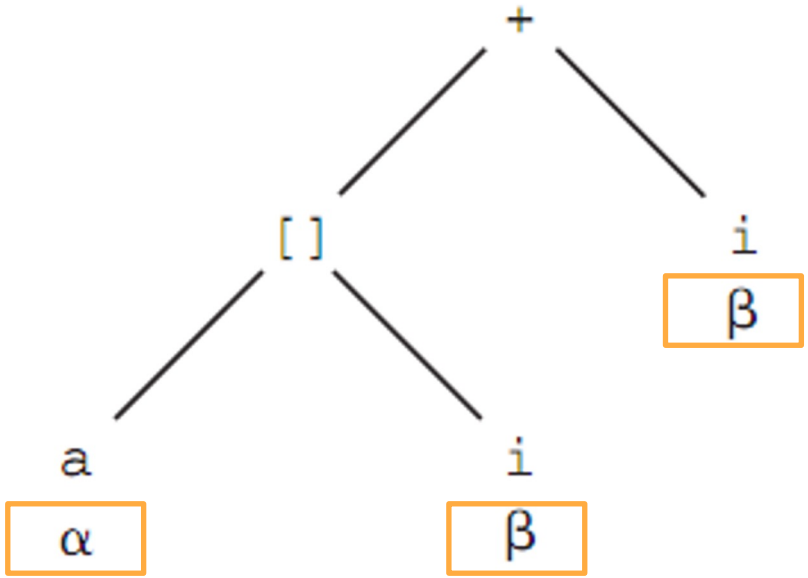


Figure 8.12 Assigning type variable to names without types

Hindley-Milner Type Checking (cont'd)

- Type checker now checks the subscript node
  - **Infers** that `a` must be an array
  - **Infers** that `i` must have an `int`
  - **Replaces** β with `int` in the entire tree

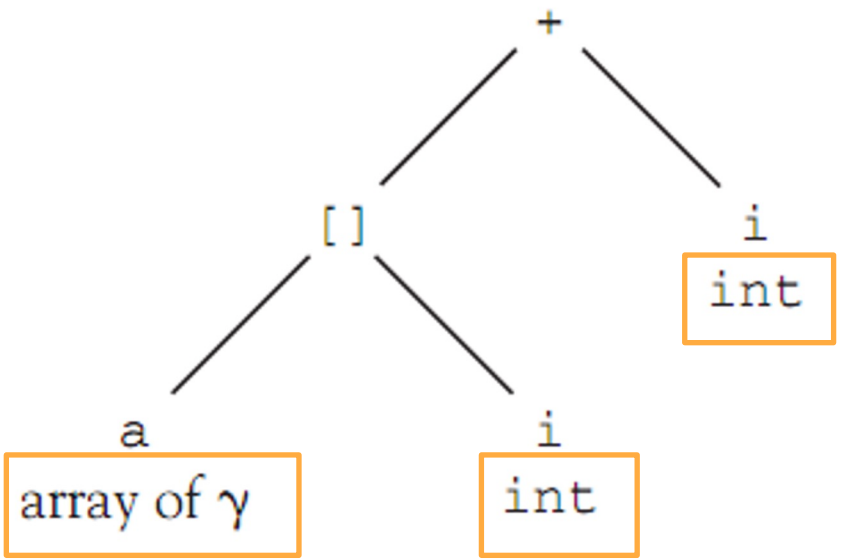


Figure 8.13 Inferring the type of the variable a

Hindley-Milner Type Checking (cont'd)

- Type checker now concludes that the **subscript node** is type correct and has the type γ

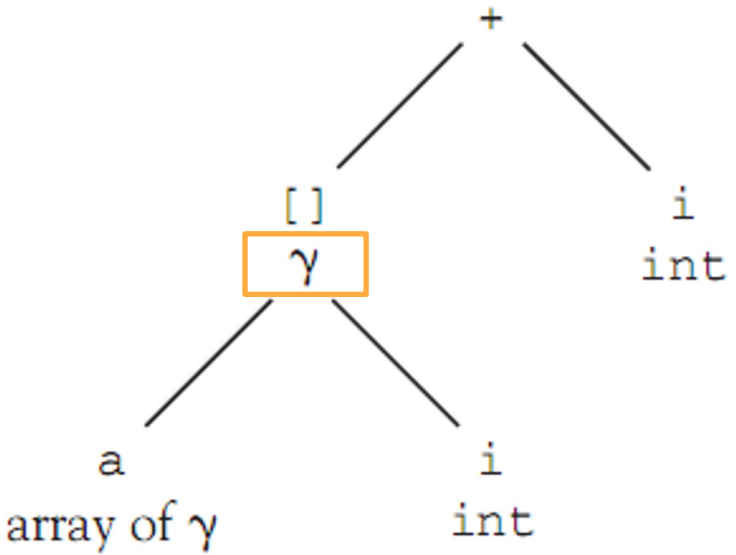


Figure 8.14 Inferring the type of the subscript node

Hindley-Milner Type Checking (cont'd)

- `+` node type is checked
  - Concludes that γ must be type `int`
  - Replaces γ everywhere by `int`

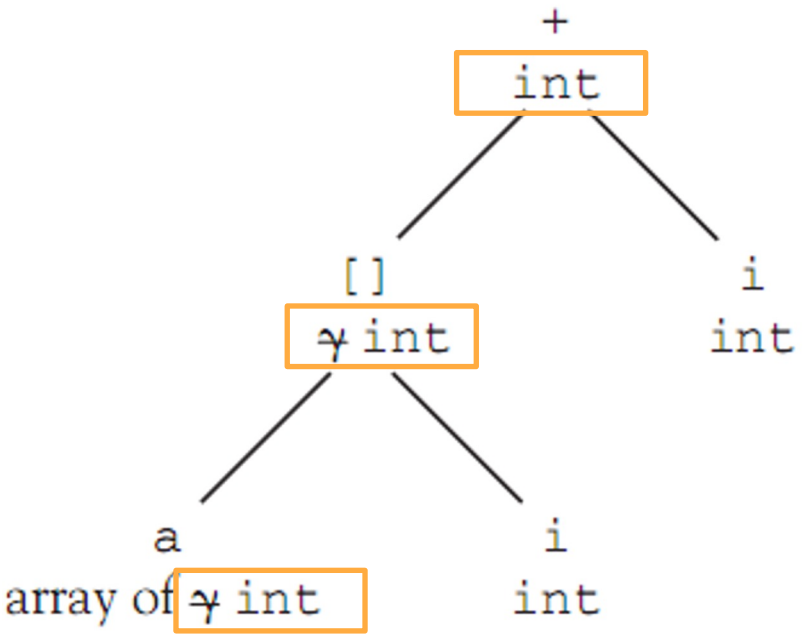


Figure 8.15 Substituting int for the type variable

Hindley-Milner Type Checking (cont’d)

- **Unification**: The process of changing type expressions for variables to ensure type checking succeeds. It's a form of pattern matching.

- **Instantiation**: When a type variable (e.g., α, β) is replaced by an actual type (e.g., int), all instances of that variable must be updated.

- Example: To check array of α against array of β, the system must unify them. It determines α == β and replaces all β with α everywhere it occurs.

Hindley-Milner Type Checking (cont’d)

- **Simplified Typing**: Reduces the amount of explicit type information the programmer must write.
- **Strong Consistency**: Allows types to remain as general as possible while still being strongly checked for consistency.
- **Implicit Polymorphism**: Automatically implements powerful parametric polymorphism.
  - **Example:** The type Array of α represents an infinite set of types (e.g., Array of int, Array of String, etc.).

References

[1] Programming languages principles and practice, Louden & Lambert, 3rd edition, Course Technology